

Tree Data Structure - Note di Studio

Indice

1. [Concetti Fondamentali](#)
2. [Binary Search Tree \(BST\)](#)
3. [Shape Property](#)
4. [Tree Traversal](#)
5. [Height](#)
6. [Problemi e Limitazioni](#)

Concetti Fondamentali

Struttura Generica di un Albero

```
class Tree<T> where T : IComparable<T>
```

- **Generico:** Funziona con qualsiasi tipo che implementa `IComparable<T>`
- **Vincolato:** Deve poter confrontare i valori
- **Root:** Nodo radice dell'albero

Nodo Base

```
public class Node {  
    public T Value;        // Valore memorizzato  
    public Node Left;      // Figlio sinistro  
    public Node Right;     // Figlio destro  
}
```

Proprietà: Ogni nodo ha massimo 2 figli → **Albero Binario**

Binary Search Tree (BST)

Definizione

Un **BST** è un albero binario dove:

- **Valore figlio sinistro** < **Valore padre**
- **Valore figlio destro** ≥ **Valore padre**

Regola di Inserimento

```
public void AddBST (T value) {
    Root = AddBST (Root, value);
}

Node AddBST (Node node, T value) {
    if (node == null)
        return new (value);

    if (value.CompareTo (node.Value) < 0)
        node.Left = AddBST (node.Left, value);    // Minore → Sinistra
    else
        node.Right = AddBST (node.Right, value); // Maggiore → Destra

    return node;
}
```

Meccanismo:

- Se il valore è **minore** → scendi a SINISTRA
- Se il valore è **maggiore o uguale** → scendi a DESTRA
- Se il nodo è `null` → crea un nuovo nodo

Complessità

Operazione	Caso Ideale	Caso Peggior
Ricerca	$O(\log n)$	$O(n)$
Inserimento	$O(\log n)$	$O(n)$
Cancellazione	$O(\log n)$	$O(n)$

Il caso peggiore si verifica quando i dati sono **già ordinati** → albero degenerato in lista

Esempio

Inserisci: 10, 5, 15, 3, 7

10

```

/  \
5   15
/  \
3   7

```

È ordinato (in-order traversal: 3, 5, 7, 10, 15)

Shape Property

Definizione

Un albero ha la **Shape Property** quando:

- Tutti i livelli sono completamente riempiti
- L'ultimo livello è riempito da **sinistra verso destra**
- = **Complete Binary Tree**

Inserimento con Shape Property (AddSh)

```

public void AddSh (T value) {
    int n = ++Count;           // Numero il nodo (1, 2, 3, ...)
    string guide = Convert.ToString(n, 2); // Converti in binario
    Root = AddSh (Root, value, guide);
}

```

Numerazione Binaria

Numero decimale → Binario → Percorso

n=1	→ 1	→ (radice)
n=2	→ 10	→ Sinistra
n=3	→ 11	→ Destra
n=4	→ 100	→ Sinistra, Sinistra
n=5	→ 101	→ Sinistra, Destra
n=6	→ 110	→ Destra, Sinistra
n=7	→ 111	→ Destra, Destra

Algoritmo SiftDown

```

Node AddSh (Node node, T value, string guide) {
    if (node == null)
        return new (value);

    guide = guide[1..]; // Rimuovi il primo bit (sempre 1)

    if (guide[0] == '0')
        node.Left = AddSh (node.Left, value, guide);
    else
        node.Right = AddSh (node.Right, value, guide);

    return node;
}

```

Meccanismo:

1. Leggi il primo bit della `guide`
2. Se `0` → vai a SINISTRA
3. Se `1` → vai a DESTRA
4. Ripeti finché non raggiungi `null` → crea nodo

Esempio

Inserisci 5 valori: 10, 20, 30, 40, 50

Count=1: <code>guide="1"</code>	→ Crea radice (10)
Count=2: <code>guide="10"</code>	→ Sinistra (20)
Count=3: <code>guide="11"</code>	→ Destra (30)
Count=4: <code>guide="100"</code>	→ Sinistra, Sinistra (40)
Count=5: <code>guide="101"</code>	→ Sinistra, Destra (50)

Albero completo:

```

10
/  \
20   30
/  \
40  50

```

Vantaggi

$O(\log n)$ per inserimento (profondità = $\log n$)

Nessun ordinamento richiesto

Perfetto per **Heap**

Nessun ribilanciamento

Limitazioni

Non mantiene ordinamento BST

Non ottimale per ricerca

Tree Traversal (Visita dell'Albero)

Tre Modalità di Attraversamento

```
public void Visit (Action<T, int> act, int level) {  
    Left?.Visit (act, level + 1);  
    act (Value, level);  
    Right?.Visit (act, level + 1);  
}
```

Nel codice è implementato: **IN-ORDER** (Sinistra → Nodo → Destra)

1° Pre-order (Radice → Sinistra → Destra)

```
public void PreOrder (Action<T> act) {  
    act (Value);  
    Left?.PreOrder (act);  
    Right?.PreOrder (act);  
}
```

Output per BST ordinato: Radice prima di tutti gli altri

Uso: Copia di alberi, espressioni prefisse

2° In-order (Sinistra → Radice → Destra)

```
public void InOrder (Action<T> act) {  
    Left?.InOrder (act);  
    act (Value);  
    Right?.InOrder (act);  
}
```

Output per BST: Valori in ordine crescente P

Uso: Ordinamento, espressioni infisse

Esempio:

```
BST:      10
/  \
5    15
```

In-order output: 5, 10, 15 (ordinato!)

3.2 Post-order (Sinistra → Destra → Radice)

```
public void PostOrder (Action<T> act) {
    Left?.PostOrder (act);
    Right?.PostOrder (act);
    act (Value);
}
```

Output per BST ordinato: Radice per ultimo

Uso: Eliminazione, espressioni postfisse

Height (Altezza)

Definizione

L'**altezza di un nodo** = distanza massima da quel nodo a una foglia

```
public int Height {
    get {
        int h1 = Left?.Height ?? 0;    // Altezza figlio sinistro
        int h2 = Right?.Height ?? 0;   // Altezza figlio destro
        return Math.Max (h1, h2) + 1;  // Max + 1 (il nodo stesso)
    }
}
```

Calcolo

- **Foglia:** Height = 1

- **Nodo con figli:** $\text{Height} = 1 + \max(\text{Left.Height}, \text{Right.Height})$

Esempio

```

10                Height(10) = 3
/  \
5    15           Height(5) = 2, Height(15) = 1
/  \
3    7            Height(3) = 1, Height(7) = 1

```

Calcolo:

- $\text{Height}(3) = 1$ (foglia)
- $\text{Height}(7) = 1$ (foglia)
- $\text{Height}(5) = 1 + \max(1, 1) = 2$
- $\text{Height}(15) = 1$ (foglia)
- $\text{Height}(10) = 1 + \max(2, 1) = 3$

Ricorsione

È **ricorsiva**: ogni nodo chiede l'altezza dei suoi figli

Base della ricorsione: foglie (`null` → 0)

Case ricorsivo: $1 + \max(\text{altezza figli})$

Problemi e Limitazioni

1. Sbilanciamento dell'Albero

Problema: Se i dati in input sono **già ordinati**, il BST degenera

Inserisci: 1, 2, 3, 4, 5 (già ordinato)

Caso peggiore:

```

1
 \
2
 \
3
 \
4
 \
5

```

$\text{Height} = 5$ (lista, non albero!)

Complessità: $O(n)$ invece di $O(\log n)$

Soluzione:

- Randomizzare i dati prima dell'inserimento
- Usare **AVL Tree** o **Red-Black Tree** (auto-bilanciamento)

2. Mancanza di Ordinamento in AddSh

Fatto: AddSh mantiene la Shape Property **ma NON l'ordine**

Inserisci random: 5, 10, 3, 20, 7

```
5
 / \
10  3
 / \
20  7
```

NON è ordinato (in-order: 10, 20, 5, 7, 3)
È completo (forma)

Uso corretto: Base per **Heap** (Priority Queue), non per ricerca

Tabella Riassuntiva

Concetto	AddBST	AddSh
Mantiene ordine	Sì	No
Shape Property	No	Sì
Complessità ideale	$O(\log n)$	$O(\log n)$
Complessità peggiore	$O(n)$	$O(\log n)$ P
Caso d'uso	Ricerca veloce	Heap, Priority Queue
Traversal In-order	Ordinato	Casuale

Concetti Chiave

Generici in C#

```
class Tree<T> where T : IComparable<T>
```

- `<T>`: Tipo generico
- `where T : IComparable<T>`: Vincolo (deve implementare confronto)
- Consente di usare la stessa classe per `int`, `string`, `double`, ecc.

Ricorsione

```
Node AddBST (Node node, T value) {  
    if (node == null) return new(value); // Base  
    if (value < node.Value) // Caso ricorsivo  
        node.Left = AddBST(node.Left, value);  
    return node;  
}
```

- **Base**: quando `node == null`
- **Caso ricorsivo**: chiama se stesso sui figli
- **Ritorno**: sempre ritorna il nodo

Null Coalescing (??)

```
int h1 = Left?.Height ?? 0;
```

- `Left?.Height`: accedi a `Height` se `Left` non è null
- `?? 0`: se il risultato è null, usa 0

Balance Factor

Definizione

Il **Balance Factor** di un nodo misura l'**equilibrio** dell'albero in quel punto:

```
Balance Factor = Height(Sottoalbero Sinistro) - Height(Sottoalbero Destro)
```

Implementazione

```
public int BalanceFactor {  
    get {  
        int leftHeight = Left?.Height ?? 0;  
        int rightHeight = Right?.Height ?? 0;  
        return leftHeight - rightHeight;  
    }  
}
```

Interpretazione dei Valori

Balance Factor	Significato	Stato
-1, 0, +1	Perfettamente bilanciato	OK (AVL valido)
> +1	Sbilanciato a SINISTRA	Sottoalbero sinistro troppo profondo
< -1	Sbilanciato a DESTRA	Sottoalbero destro troppo profondo

Calcolo Passo-passo

Albero:

```
10  
 /  \  
5    15  
 /  \  
3    7
```

Nodo 3 (foglia):
BalanceFactor = 0 - 0 = 0

Nodo 7 (foglia):
BalanceFactor = 0 - 0 = 0

Nodo 5 (ha figli):
BalanceFactor = Height(3) - Height(7) = 1 - 1 = 0

Nodo 15 (foglia):
BalanceFactor = 0 - 0 = 0

Nodo 10 (radice):

$\text{BalanceFactor} = \text{Height}(5) - \text{Height}(15) = 2 - 1 = 1$

Ricorsione nel Calcolo

Calcola 10.BalanceFactor

```
|
|— Calcola Left.Height (5)
|   |— Calcola Left.Height (3) → 1
|   |— Calcola Right.Height (7) → 1
|   return 1 + max(1, 1) = 2
|— Calcola Right.Height (15) → 1
```

return 2 - 1 = 1 ← BalanceFactor di 10

Uso nei Self-Balancing Trees

AVL Tree:

- Dopo ogni inserimento/cancellazione, controlla il Balance Factor
- Se $|\text{BF}| > 1$, esegui rotazioni per ribilanciare
- Garantisce $O(\log n)$ per tutte le operazioni

Red-Black Tree:

- Non usa esattamente il Balance Factor
- Usa colori per mantenere l'equilibrio

Esempio: Sbilanciamento

Inserisci ordinato: 1, 2, 3, 4, 5

```
1
 \
2   BalanceFactor(1) = 0 - 1 = -1
 \
3   BalanceFactor(1) = 0 - 2 = -2  Sbilanciato!
 \
4
 \
5
```

Necessarie rotazioni per ribilanciare

Priority Queue da Min Heap

Concetto Fondamentale

Una **Priority Queue** implementata con **Min Heap** è una struttura dati dove:

```
Elementi hanno una priorità (il valore stesso)
L'elemento con priorità MINIMA esce PRIMA
Non è FIFO (First In First Out)
È una coda basata su priorità
```

Proprietà del Min Heap

```
public class MinHeapPriorityQueue<T> where T : IComparable<T> {
    private List<T> items = new ();

    // Min Heap Property: Padre < Figli
    // Valore minore = Priorità massima
}
```

Regola fondamentale:

- **Ogni padre $\leq$ dei suoi figli**
- Il minimo è sempre alla radice (indice 0)

Operazioni Principali

1. Insert (SiftUp)

```
public void Insert (T value) {
    items.Add (value);           // Aggiungi alla fine (mantieni shape)
    SiftUp (items.Count - 1);    // Risali se minore del padre
}

private void SiftUp (int index) {
    while (index > 0) {
        int parentIndex = (index - 1) / 2;

        if (items[index].CompareTo (items[parentIndex]) < 0) {
            // Scambia se minore del padre
        }
    }
}
```

```

(items[index], items[parentIndex]) =
(items[parentIndex], items[index]);
index = parentIndex;
}
else {
break; // Heap property ripristinata
}
}
}

```

Complessità: $O(\log n)$

2. ExtractMin (SiftDown)

```

public T ExtractMin () {
    if (items.Count == 0)
        throw new Exception ("Coda vuota");

    T min = items[0];                // Estrai il minimo (radice)
    items[0] = items[items.Count - 1]; // Sposta ultimo alla radice
    items.RemoveAt (items.Count - 1); // Rimuovi ultimo

    if (items.Count > 0)
        SiftDown (0); // ← Questo ripristina la heap property

    return min;
}

```

3. SiftDown (Algoritmo Chiave)

```

private void SiftDown (int index) {
    while (true) {
        int smallest = index;
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;

        // Trova il figlio più piccolo
        if (leftChild < items.Count &&
            items[leftChild].CompareTo (items[smallest]) < 0)
            smallest = leftChild;

        if (rightChild < items.Count &&
            items[rightChild].CompareTo (items[smallest]) < 0)
            smallest = rightChild;
    }
}

```

```
// Se il più piccolo non è il nodo corrente, scambia
if (smallest != index) {
  (items[index], items[smallest]) =
  (items[smallest], items[index]);
  index = smallest;  // Continua scendendo
}
else {
  break;  // Heap property ripristinata!
}
}
```

Complessità: $O(\log n)$

Perché è una Priority Queue?

CODA FIFO NORMALE:
Inserisci: A → B → C → D
Estrai: A, B, C, D (ordine di inserimento)

MIN HEAP PRIORITY QUEUE:
Inserisci: 10, 5, 20, 3
Estrai: 3, 5, 10, 20 (ordine di priorità!)
↑ Priorità minima esce PRIMA

Il valore stesso è la priorità:

- Valore minore = Priorità **massima**
- Valore maggiore = Priorità **minima**

Esempio Pratico: Pronto Soccorso

```

var pronto = new MinHeapPriorityQueue<int>();

// Priorità: 1=Critico, 5=Serio, 10=Lieve
pronto.Insert(10); // Paziente con febbre (lieve)
pronto.Insert(1);  // Paziente con infarto (CRITICO!)
pronto.Insert(5);  // Paziente con frattura (serio)

// Ordine di visita:
pronto.ExtractMin(); // 1 (CRITICO - esce PRIMA!)
pronto.ExtractMin(); // 5 (serio)
pronto.ExtractMin(); // 10 (lieve)

```

Visualizzazione SiftDown Passo-Passo

HEAP INIZIALE (Min Heap):

```

3
/ \
8  20
/ \ /
5  15 10

```

Rimuoviamo 3 (radice, esce con priorità massima):

Passo 1: Sposta 10 alla radice

```

10          ← Viola! 10 > figli
/ \
8  20
/ \
5  15

```

Passo 2: SiftDown(0) - Scambia con figlio minore (8)

```

8          ← OK ora
/ \
10  20
/ \
5  15

```

Passo 3: SiftDown(1) - Controlla 10

10 vs 5,15 → 5 è minore

Scambia 10 con 5:

```

8
/ \
5  20

```

```
/ \
10 15    ← Heap property ripristinata!
```

Confronto: Insert vs ExtractMin

Operazione	Complesso	Note
Insert (SiftUp)	$O(\log n)$	Sale fino alla radice se necessario
ExtractMin (SiftDown)	$O(\log n)$	Scende fino a foglia se necessario
Peek (leggi minimo)	$O(1)$	items[0] - nessun movimento

Rappresentazione in Array

```
Min Heap: [3, 8, 20, 5, 15, 10]
Index:    0  1  2  3  4  5
```

Relazioni indici:

- Padre di $i \rightarrow (i-1)/2$
- Figlio sx di $i \rightarrow 2*i + 1$
- Figlio dx di $i \rightarrow 2*i + 2$

Esempio: indice 1 (valore 8)

- Padre: $(1-1)/2 = 0 \rightarrow 3$ ($8 > 3$, ok)
- Figlio sx: $2*1+1 = 3 \rightarrow 5$ ($8 > 5$, ok)
- Figlio dx: $2*1+2 = 4 \rightarrow 15$ ($8 < 15$, ok)

Quando Usare Min Heap Priority Queue

Algoritmi di routing (Dijkstra: estrai il nodo con distanza minima)

Scheduling (estrai il task con priorità massima)

Heap Sort (ordina estraendo i minimi)

Huffman Coding (compressione dati)

Gestione code di attesa (pazienti per gravità)

*A search** (percorsi con costo minimo)

Max Heap Priority Queue

Per estrarre il **massimo** prima del minimo, usa **Max Heap**:

```
// Max Heap Property: Padre > Figli
// Valore maggiore = Priorità massima

if (items[index].CompareTo (items[parentIndex]) > 0) { // > invece di <
    (items[index], items[parentIndex]) =
    (items[parentIndex], items[index]);
}
```

Collegamento: AddSh → Heap → Priority Queue

```
AddSh (Shape Property)
↓
Mantiene forma completa dell'albero
↓
Min Heap Priority Queue
↓
Proprietà aggiuntiva: Padre < Figli
↓
Insert (SiftUp) + ExtractMin (SiftDown)
↓
Priority Queue funzionante!
```

La tua **AddSh** è il primo passo per costruire un Heap!

Correlazioni con altri concetti

- **BST** → Base per ricerca efficiente
- **Shape Property** → Base per **Heap**
- **Heap** → Base per **Priority Queue** e **Heap Sort**
- **Tree Traversal** → Utile per **DFS** (Depth-First Search)
- **Height** → Misura l'equilibrio (AVL, Red-Black)
- **Generici** → Pattern per codice riutilizzabile

Riferimenti nel Codice

Linea	Concetto
Visit(Action<T, int> act)	

In-order traversal generico	
AddBST(Node node, T value)	Inserimento ordinato
AddSh(Node node, T value, string guide)	Inserimento con shape property
Height { get {...} }	Calcolo ricorsivo altezza
tree.Visit(Print)	Stampa con indentazione (livello)